BITEBOX

Project Documentation

Food delivery platform: customer, restaurant, delivery and admin panels

Angular 22 · Node.js / Express 5 · MongoDB

Complete reference: architecture, configuration, roles and permissions, data model, business rules, realtime, PWA and push, API reference, screens, security, testing and deployment.

10 November 2026

This is the full reference for the BiteBox food delivery platform: what it does, how it is built, every API endpoint, the data model, the business rules, and how to run, test and deploy it.

- **Quick start:** README.md.
- **Problems and fixes:** TROUBLESHOOTING.md.
- **Coins and rewards design notes:** COINS_REWARDS_PLAN.md.
- **Live API contract:** Swagger UI at `/api/docs`, generated from `backend/docs/openapi.*.js`. A test fails if a route is missing from it.

Contents

1. Overview
2. Technology stack
3. Repository layout
4. Getting started
5. Configuration reference
6. Backend architecture
7. Frontend architecture
8. Users, roles and permissions
9. Data model
10. Business rules
11. Realtime (Socket.IO)
12. Notifications and Web Push
13. Progressive Web App
14. API reference
15. Screens and routes
16. Security
17. Performance and accessibility
18. Testing
19. Deployment
20. Conventions for contributors

1. Overview

BiteBox connects four groups of people, and each one has its own panel in one web app:

Panel	Who	What they do
Customer site (`/`)	Customers	Find restaurants near a delivery point, order and pay, track the rider live, rate the order, earn and spend coins, refer friends, raise support tickets
Restaurant panel (`/restaurant`)	Restaurant partners	Set up the restaurant and menu, accept and prepare orders, reply to reviews, see earnings
Delivery panel (`/delivery`)	Riders (delivery partners)	Go online, accept delivery offers, share live GPS, complete deliveries, see earnings and ratings
Admin panel (`/admin`)	Super admin plus operations, finance and support staff	Approve restaurants, run orders and riders, handle money, support tickets, reviews, rewards and promotions

How an order works, end to end:

1. The customer picks a delivery point. The site lists approved restaurants within 15 km.
2. They build a cart. The server prices everything: items, coupon, coins, delivery fee, platform fee and GST.
3. They place the order with **cash on delivery** or **online payment** (Razorpay, or a simulated test payment in development).
4. The restaurant accepts it, prepares it and marks it ready. If it does not answer within 10 minutes, the order is rejected automatically.
5. Dispatch offers the order to the nearest eligible rider, one rider at a time, each with 90 seconds to answer.
6. The rider picks it up and delivers it. The customer watches the rider move on a map.
7. On delivery, the money split is fixed: restaurant, rider and platform. The customer earns coins and can rate the restaurant, the dishes and the rider.

Every status change is pushed live to each screen involved, saved as an in-app notification, and sent as a browser push notification when the person has turned those on.

2. Technology stack

Layer	Technology
Frontend	Angular 22: standalone components, signals, `OnPush`, zoneless, SSR with prerendering, Signal Forms
UI	Angular Material 22 (M3) and NexPrime (`nexprime`), SCSS, Lucide icons through NexPrime
Maps	Leaflet with OpenStreetMap tiles. Nominatim for geocoding and OSRM for road routing, both called through the API.
PWA	`@angular/service-worker`, a web manifest, Web Push (VAPID)
Backend	Node.js (ES modules), Express 5, Mongoose 9, express-validator, Socket.IO 4, pino
Auth	JWT access tokens (HS256) plus rotating refresh tokens in an HttpOnly cookie. Passwords hashed with bcrypt.
Payments	Razorpay, with a simulated provider for development
Images	Cloudinary, falling back to local disk
Email	Nodemailer over SMTP. In development the reset link is logged instead.
Database	MongoDB 7+ (local, Docker or Atlas). Uses transactions when MongoDB runs as a replica set.
Tests	Backend: `node:test`, supertest, mongodb-memory-server. Frontend: Vitest with jsdom.
Delivery	Docker Compose (MongoDB replica set, API, SSR web server, nginx), GitHub Actions CI. The frontend also runs on Vercel or Netlify.

Versions: Node.js 24 LTS (or 22.22+ / 26+, as Angular 22 requires), npm 10+, MongoDB 7+.

3. Repository layout

```
backend/                    Express API (JavaScript ES modules, no src/ folder)
  server.js                 HTTP server + Socket.IO, DB connection, background loops, graceful shutdown
  app.js                    Express app factory: middleware, route mounting, error handling
  api/index.js              Serverless entry (Vercel)
  config/                   env, logger, db, cloudinary, roles, permissions,
                            business rules: pricing, dispatch, earnings, reviews, rewards
  routes/                   one router file per area (mount points in app.js)
  validators/               express-validator rule sets
  controllers/              thin HTTP handlers for the older areas (auth, catalogue, checkout, orders)
  services/                 all business logic (see §6)
  services/payments/        payment.service + razorpay and mock providers
  models/                   Mongoose models (see §9)
  sockets/                  Socket.IO: handshake, rooms, tracking, event emitters
  middleware/               auth (authenticate/authorize/can), validate, security, errors
  utils/                    ApiError, respond, transactions, geo, dates, pagination, upstream, sweep loop…
  docs/                     OpenAPI fragments merged by docs/openapi.js
  scripts/seed.js           idempotent development data
  tests/                    integration tests, one file per area
  Dockerfile                production image

frontend/                   Angular 22 app
  src/app/
    app.ts, app.html        root component (page loader, deferred app-status)
    app.config.ts           providers: router, HTTP + interceptors, hydration, service worker
    app.routes.ts           top-level routes (panels are lazy)
    app.routes.server.ts    SSR render mode per route
    core/                   auth service, guards, interceptors, API and app services
    shared/                 components, forms helpers, models, icons, pipes, list helpers
    layout/                 customer, auth, admin, restaurant, delivery layouts + panel-shell
    features/               screens per area (see §15)
  src/server.ts             SSR Express server (security headers, compression, caching, /healthz)
  src/environments/         apiUrl per build (no secrets)
  public/                   static assets: icons, manifest, _headers, _redirects
  ngsw-config.json          service worker caching rules
  Dockerfile                production image

deploy/                     nginx.conf and the .env template for docker compose
docker-compose.yml          full production-like stack
vercel.json                 static hosting of the frontend with /api proxy
.github/workflows/ci.yml    lint, tests and build on every push and pull request
docs/                       this documentation, troubleshooting, coins and rewards plan
```

4. Getting started

Install

```
cd backend  && npm install && cp .env.example .env   # then fill in .env (see §5)
cd ../frontend && npm install
```

Seed development data (optional)

```
cd backend
SEED_PASSWORD='Choose-a-dev-pass1' npm run seed   # safe to rerun; add -- --reset-passwords to reset them
```

The seed refuses to run when `NODE_ENV=production` and never creates duplicates. Every account uses `SEED_PASSWORD`:

Role	Email
Super admin	`admin@bitebox.test`
Operations admin	`operations@bitebox.test`
Finance admin	`finance@bitebox.test`
Support admin	`support@bitebox.test`
Customer	`customer@bitebox.test`
Restaurant partner	`restaurant@bitebox.test`
Rider	`rider@bitebox.test`

It also creates:

- 8 food categories.
- The approved restaurant *Spice Route Kitchen* (Indiranagar, Bengaluru) with 8 dishes.
- The coupons `WELCOME50` (50% off up to ₹100, minimum ₹199, once per customer) and `FLAT75` (₹75 off orders of ₹399 or more).
- A default address for the customer, about 4 km from the restaurant.
- A motorbike for the rider, who starts offline.

All seed data is fictional.

Run

```
cd backend  && npm run dev   # API on http://localhost:5000 (Swagger: /api/docs, health: /api/v1/health)
cd frontend && npm start     # web app on http://localhost:4200
```

Use the npm scripts, not a bare `ng`. They set `NG_BUILD_SASS_EMBEDDED=0`, because the embedded Sass process can hang the build on some machines.

Signing in: every role signs in at `/login` and lands in its own panel. The old `/admin/login`, `/restaurant/login` and `/delivery/login` addresses redirect there. Sign-up is separate: `/register` (customers), `/restaurant/register` and `/delivery/register`. Admin accounts cannot sign up; the super admin creates them.

Try the whole order flow

Use four browser profiles, or normal and private windows.

1. **Rider:** go online. This needs a precise location. On a desktop, set one in Chrome DevTools → Sensors, for example `12.9729, 77.6412`, next to the seeded restaurant.
2. **Customer:** order from *Spice Route Kitchen*.
3. **Restaurant:** open Orders, then Accept → Start preparing → Ready for pickup.
4. **Rider:** accept the offer, then Picked up → Start delivery → Delivered.
5. **Customer:** watch each step on the order page, then rate the order. **Admin:** check `/admin/orders`.

5. Configuration reference

Backend (`backend/.env`, template in `.env.example`)

`.env` is git-ignored. Never commit real values. In tests, the external keys (Cloudinary, VAPID) are always ignored.

Variable	Default	Purpose
`NODE_ENV`	`development`	`production` enables secure cookies and info-level logging, and disables the test payment and the seed
`PORT`	`5000`	API port
`MONGO_URI`	(required)	MongoDB connection string
`CLIENT_URL`	`http://localhost:4200`	Allowed browser origins, comma-separated. The first one is used in emails.
`JWT_SECRET`	(required, 32+ chars)	Signs access tokens
`JWT_EXPIRES_IN`	`15m`	Access token lifetime
`REFRESH_TOKEN_TTL_DAYS`	`7`	Refresh session lifetime
`PASSWORD_RESET_TTL_MINUTES`	`30`	Reset link lifetime
`BCRYPT_ROUNDS`	`12` (`4` in tests)	Password hashing cost
`COOKIE_SAMESITE`	`strict`	Set `none` (HTTPS only) when the web app and API are on different sites
`TRUST_PROXY`	unset	Set `1` behind a reverse proxy, so rate limits see real client IPs
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `EMAIL_FROM`	empty	Password-reset email. If empty in development, the link is printed to the console.
`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`, `CLOUDINARY_FOLDER`	empty, `bitebox`	When all three keys are set, images go to Cloudinary
`UPLOAD_DIR`, `PUBLIC_BASE_URL`	`uploads`, `http://localhost:5000`	Local image storage and the origin used in image URLs
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`	empty	Enable Razorpay online payment
`RAZORPAY_WEBHOOK_SECRET`	empty	Verifies `/payments/webhooks/razorpay`
`PAYMENTS_MOCK`	on outside production	`false` hides the simulated payment. It is never available in production.
`GEOCODER_URL`, `GEOCODER_USER_AGENT`, `GEOCODER_COUNTRY_CODES`	Nominatim, generic, `in`	Place search. Put a contact email in the user agent, as Nominatim requires.

Variable	Default	Purpose
`ROUTING_URL`, `ROUTING_PROFILE`	public OSRM, `driving`	Road routes and travel-time estimates
`REPORT_TIMEZONE`	`Asia/Kolkata`	Day boundaries for reports, earnings and birthdays
`VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`	empty	Web Push is on only when both are set (`npx web-push generate-vapid-keys`)
`VAPID_SUBJECT`	`mailto:support@bitebox.local`	Contact address for the push services
`PUSH_ALLOWED_HOSTS`	FCM, Mozilla, Windows, Apple	Push services a subscription may point at (SSRF guard)
`DISABLE_TRANSACTIONS`	unset	`true` forces the undo-based fallback instead of MongoDB transactions
`LOG_LEVEL`, `LOG_HTTP`	`debug` / `info` / `silent`	Logging. `LOG_HTTP=true` logs every request.
`SEED_PASSWORD`	(none)	Password for the seeded accounts
`TEST_MONGO_URI`	unset	Tests use a real database instead of the in-memory one. The name must end in `_test`.

Frontend

- `src/environments/environment.ts` (development) calls the API on port 5000 of the page's own host, so it also works from a phone on your LAN. Add that page origin to `CLIENT_URL`.
- `src/environments/environment.prod.ts` uses `apiUrl: '/api/v1'`, which assumes a proxy on the same origin.
- The SSR server reads three variables:
  - `PORT` (default 4000).
  - `API_INTERNAL_URL`: how the server reaches the API during rendering.
  - `NG_ALLOWED_HOSTS`: your domain, needed by Angular's SSRF protection.

These files are bundled into browser code. **Never put secrets in them.**

Business rule settings (code, not env)

File	Contents
`config/pricing.js`	Delivery fee, free-delivery threshold, range, platform fee, GST
`config/dispatch.js`	Offer timeout, pickup radius, location freshness, payment window, restaurant response time, pickup window, sweep interval
`config/earnings.js`	Restaurant commission, rider pay
`config/reviews.js`	Review windows, lengths, photos, report reasons

File	Contents
`config/rewards.js`	Default coin, referral, birthday and welcome settings. The admin can change them at runtime.
`config/permissions.js`	Admin permissions per role

6. Backend architecture

Request flow

```
request → helmet / cors / compression → json (100 kB) → cookie-parser → mongoSanitize
        → router → authenticate → authorize(...roles) | can(PERMISSION) → rate limit
        → validate(rules) → controller/handler → service → Mongoose model
        → respond(res, status, message, data)   or   throw ApiError → errorHandler
```

- `validate` (express-validator) replaces `req.body` with only the validated fields (`matchedData`) and puts the cleaned query in `req.validatedQuery`. Unknown fields never reach a service.
- **Every response** uses the same envelope:

  ```
  { "success": true,  "message": "Signed in successfully", "data": { } }
  { "success": false, "message": "Validation failed", "error": { "code": "VALIDATION_ERROR", "details": [{
  "field": "email", "message": "..." }] } }
  ```

  Throw `ApiError` (`utils/api-error.js`) for errors the client should see. Anything else becomes a generic 500, and the stack is only logged.
- **Money** is always integer **paise** (₹1 = 100). `pricePaise: 32900` is ₹329.
- **The Razorpay webhook** is mounted *before* the JSON parser, because its signature covers the raw body.

Services

Area	Services
Accounts	`auth.service` (register, login, sessions, reset), `admin-staff.service`, `audit.service`
Catalogue	`restaurant.service`, `food.service`, `category.service`, `upload.service`
Checkout	`cart.service` (server-priced quote), `pricing.service`, `coupon.service`, `address.service`, `order.service#placeOrder`
Orders	`order-workflow.service` (the **only** place a status changes), `order-management.service` (role-specific views), `order-notifications`
Dispatch	`dispatch.service` (matching, offers, `sweep()`), `delivery-partner.service`
Payments	`payments/payment.service` (owns every payment state change), `razorpay.provider`, `mock.provider`
Money	`earnings.service#settlementFor`, `reports.service`, `finance.service` (payouts, refunds, commission)
Operations and support	`operations.service`, `support.service`
Reviews	`review.service` (rating rebuilds), `review-moderation.service`

Area	Services
Rewards	`coin-wallet.service` (the only ledger writer), `coins.service`, `referral.service`, `special-rewards.service`, `reward-config.service`, `reward-events`, `rewards-jobs.service`, `rewards-admin.service`
Messaging	`notification.service#notify`, `push.service`, `mail.service`, `newsletter.service`
Geo	`geocoding.service` (Nominatim), `routing.service` (OSRM). Both use `utils/upstream.js`, which caches, merges identical requests and allows about 1 call per second.

Background loops (started in `server.js`)

Loop	Every	Work
`dispatch.service#sweep`	15 s	Expire rider offers, re-dispatch waiting orders, auto-reject unanswered orders, cancel unpaid online orders and orders never picked up
`rewards-jobs.service#rewardsSweep`	2 min	Finish pending ledger rows, settle coin holds, expire coins and referral windows, start and end campaigns, pay birthdays

Both are idempotent and safe to run on several API instances at once. Tests call `sweep(now)` and `rewardsSweep(now)` directly.

Multi-document writes

`utils/transaction.js#withTransaction(work)` runs `work(session)` in a MongoDB transaction on replica sets (Atlas, docker-compose). On a standalone `mongod` it calls `work(null)`, and the caller must undo completed steps itself; `order.service.js#placeOrder` is the reference pattern. Both paths are tested. Set `DISABLE_TRANSACTIONS=true` to force the fallback.

Startup and shutdown

`server.js` does the following:

1. Connects to MongoDB and attaches Socket.IO.
2. Backfills `order.submittedAt` on old orders.
3. Starts both loops.
4. On `SIGTERM` / `SIGINT`, shuts down gracefully: it stops the loops, closes sockets and HTTP, then disconnects the database, and forces an exit after 10 s.

7. Frontend architecture

- **App shape:** standalone components, signals, `OnPush`, no zone.js. Every component has separate `.ts`, `.html` and `.scss` files.
- **Routing:** feature routes are lazy (`features/*/…routes.ts`). Only `/offline` is eager, so it can open without a network.
- **Render modes** (`app.routes.server.ts`):
  - The home page is rendered on the server per request.
  - `/login`, `/register` and `/forgot-password` are prerendered.
  - Everything else renders in the browser.
  - Browser-only APIs (`window`, `localStorage`, `navigator`) are only touched inside `isPlatformBrowser` / `afterNextRender`, and Leaflet is imported there dynamically.

Core services (`core/`)

Service	Role
`auth/auth.service`	Restores the session at startup via `/auth/refresh`. Holds the access token in memory and exposes `user`, `initialised` and `displayUser`. `displayUser` is a display-only hint and is never used for access decisions.
`guards/auth.guards`	`guestGuard`, `roleGuard(...roles)`, `permissionGuard(perm)`, `customerSiteGuard`
`interceptors/auth.interceptor`	Adds the bearer token, sends cookies only to `/auth/*`, refreshes once on a 401
`interceptors/server-api.interceptor`	During SSR, rewrites API calls to `API_INTERNAL_URL`
`realtime.service`	The Socket.IO client. Connects while signed in and exposes `orderUpdates$`, `track(orderId)` and `sendRiderLocation()`.
`poll.ts`	`liveRefresh` / `pollWhileVisible`: slow fallback polling that pauses on hidden tabs
`page-loader.ts`	Top progress bar for requests slower than 200 ms. `inBackground(req$)` opts a request out.
`inbox.service`	The header notification bell
`notification.service`	Snackbar helper. It is not the inbox.
`payment.service`	`pay(orderId, session)` opens Razorpay checkout.js or the mock payment dialog
`restaurant-order-board`	Shared, ref-counted live order store for the restaurant panel
`rider-gps.service`	Shares the rider's device GPS
`delivery-location.service`, `geolocation.service`, `routing.service`	Delivery point, device location, road routes

Service	Role
`cart.service`, `cart-actions.service`, `checkout-api.service`, `catalog.service`, `review.service`, `rewards.service`, `support.service`, `upload.service`, `newsletter.service`, `my-restaurant.service`, `order-management.service`, `reports.service`	API wrappers
`admin/*`	Admin API wrappers: catalogue, users, staff, finance, operations, rewards, push
`connection-status`, `app-update`, `install-prompt`, `push-notification`	PWA (see §13)
`browser-storage`, `safe-url`, `idempotency`, `http-params`, `api-error`, `current-url`	Small shared helpers

Shared UI (`shared/`)

- **Components:** maps (`map-picker`, `tracking-map` with `deliveryTracking(order)`), `daily-chart` (chart with a "Show as table" view), `stat-tile`, `notification-bell`, `confirm-dialog` (`confirmed(dialog, data)`), reviews (`review-card`, `review-form-dialog`, `public-reviews`, `rating-summary`, `stars`, `review-reply`, `report-review-dialog`), uploads (`image-upload`, `photo-upload`, `photo-viewer`), `phone-input`, `place-search`, `location-picker`, `address-form-dialog`, `bill-summary`, `qty-stepper`, `order-status-chip`, `restaurant-card`, `veg-badge`, `page-header`, `empty-state`, `site-footer`, `brand-logo`, `mock-payment-dialog`, and PWA (`app-status`, `install-banner`, `push-settings`).
- **Forms:** Signal Forms (`@angular/forms/signals`):
  - Build forms with `form(model, schema, { submission: { action } })`, `<form [formRoot]>` and `[formField]`.
  - Shared rules are in `shared/forms/form-rules.ts`. `shownError()` shows an error once the field is touched, and `serverErrors()` maps API 422 errors onto fields.
  - `NP_FORM_FIELDS` bundles the NexPrime inputs and helpers.
- **Lists:** `shared/list/paged-list.ts#pagedList(fetch, opts)` handles paging, filters, loading and errors, and live refresh.
- **Icons:** `<np-icon name="…">` with Lucide names. Material names stored in data go through `| npIcon`.
- **Styles:**
  - Brand tokens `--bb-*` live in `src/styles.scss`, with NexPrime's `--ui-*` variables mapped onto them.
  - Breakpoint mixins: `phone-only`, `tablet-up`, `below-desktop` and `desktop-up`, at 600 and 1024 px.
  - NexPrime 0.1.5 declares Angular 21 as its peer. An npm `overrides` entry pins only its Angular peers to the project's version. Remove it once NexPrime supports Angular 22.

Responsive layout

- **Mobile-first:** layouts are built for phones first.
- **Customer site:** phones get a bottom tab bar.
- **Partner and admin panels:** a persistent sidebar on desktop and a drawer below 1024 px.

- **Admin tables:** become stacked cards on narrow screens.
- **Restaurant grids:** use 1 to 4 columns.
- **Safe areas:** layouts respect iOS safe-area insets.

8. Users, roles and permissions

Roles (`config/roles.js`)

Role	Label	Signs up?	Panel
`customer`	Customer	`/register`	Customer site
`restaurant`	Restaurant partner	`/restaurant/register`	`/restaurant`
`delivery`	Delivery partner	`/delivery/register`	`/delivery`
`admin`	Super admin	no	`/admin`
`operations_admin`	Operations admin	no (created by the super admin)	`/admin/operations`
`finance_admin`	Finance admin	no	`/admin/finance`
`support_admin`	Support admin	no	`/admin/support`

User accounts are `active` or `suspended`. A suspension, password change or admin role change takes effect on the next request, because `authenticate` reloads the user and checks `status` and `tokenVersion` every time.

Admin permissions (`config/permissions.js`)

The super admin has every permission. The API checks permissions with `can(PERMISSION)`. The web app receives them in `user.permissions` and uses them only to shape menus and guards.

Permission	Allows	Ops	Finance	Support
`admins:manage`	Create, edit, activate and deactivate staff admins			
`users:manage`	List, suspend and reactivate every account			
`audit:view`	Audit log			
`catalog:manage`	Categories and coupons			
`operations:view`	Operations dashboard, monitoring, delivery performance	✓		
`orders:view`	Read orders (and receive live order events)	✓		✓
`orders:manage`	Cancel and unassign orders	✓		
`restaurants:manage`	Onboarding, approval, menu items	✓		
`riders:manage`	Suspend and reactivate riders	✓		
`finance:view`	Finance dashboard, transactions, reports, commission		✓	
`refunds:manage`	Retry refunds		✓	
`payouts:manage`	Create, pay and cancel payouts		✓	
`support:manage`	Support tickets			✓
`reviews:moderate`	Hide, remove and restore reviews; dismiss reports			✓

Permission	Allows	Ops	Finance	Support
`rewards:manage`	Change coin, referral and campaign settings			
`rewards:view`	Rewards reports and lists			
`promotions:send`	Push offers to opted-in customers			

To add a permission, add it in `config/permissions.js` and to the frontend `Permission` type (`shared/models/user.models.ts`). Use `can(...)` on the route rather than checking roles. Admin routes that change data also get `audited(action, resourceType, describe)`, which writes the audit log after a successful response.

Admin management rules:

- Super admins cannot be created or edited through the API.
- Nobody can change their own role.
- A role change signs that admin out.

Data scoping

- Partner and rider endpoints (`/my/restaurant/*`, `/delivery/*`) take their restaurant or rider from `req.user`, never from ids the client sends.
- Customers reach only their own orders, addresses, payments, tickets and notifications. Anything else returns 404.
- Each role gets its own view of an order (`order-management.service`):

View	Who	Shows
`presentForRestaurant`	Restaurant	The customer's name and area, never their phone or exact address
`presentForRider`	Rider	The full drop-off address and phone, only once the rider is carrying the order
`presentWithRider`	Customer and admin	The order plus rider details

9. Data model

All collections are Mongoose models in `backend/models/`. Money fields end in `Paise`. Locations are GeoJSON points with 2dsphere indexes (`utils/geo.js`: `toPoint`, `fromPoint`, `haversineMeters`).

Model	Key fields	Notes
User	name, email (unique), phone (unique), role, status, passwordHash, tokenVersion, referralCode, birthDate, pushPreferences	`toJSON` strips secrets. Admins also get `permissions`.
Session	user, tokenHash (unique), family, expiresAt, revokedAt	One row per refresh token. TTL index removes expired sessions.
Restaurant	owner (unique), name, slug, cuisines, address, location, costForTwoPaise, minOrderPaise, openingTime, closingTime, timezone, isAcceptingOrders, status, statusReason, rating	Status: `pending`, `approved`, `rejected`, `suspended`
FoodCategory	name, slug, icon, imageUrl, sortOrder, isActive	
FoodItem	restaurant, category, name, pricePaise, isVeg, isAvailable, imageUrl, prepTimeMinutes, tags, rating, deletedAt	Soft delete (`deletedAt`)
Address	user, label, recipient, phone, lines, city, postalCode, location, isDefault	At most 10 per customer
Cart	user (unique), restaurant, items, couponCode, coinsToRedeem	One cart, one restaurant
Coupon / **CouponRedemption**	code, discountType, discountValue, maxDiscountPaise, minOrderPaise, validity, usageLimit, usedCount, perUserLimit, restaurant / coupon, user, order	Usage is reserved atomically
Order	orderNumber, customer, restaurant, snapshots (restaurant, items, address), pricing, coupon, coins, distanceMeters, status, statusHistory, paymentMethod, paymentStatus, submittedAt, deliveryPartner, offer, declinedBy, cancellation, settlement, payouts, idempotencyKey	Statuses: see §10.3. Status groups (`ACTIVE_STATUSES`, `FINAL_STATUSES`, `TRACKED_STATUSES`) are defined next to `ORDER_STATUS`.
Payment	order (unique), user, method, provider, amountPaise, status, provider ids, failureReason, paidAt, refundedAt, processedEvents	`processedEvents` makes webhooks apply once
DeliveryPartner	user (unique), vehicleType, vehicleNumber, isOnline, location, locationUpdatedAt, locationAccuracy, currentOffer, activeOrder, lastAssignedAt, completedDeliveries, rating	`currentOffer` and `activeOrder` are compare-and-set locks
Payout	type (restaurant or rider), payee, orders, amountPaise, period, status, reference, paidBy, paidAt	

Model	Key fields	Notes
Notification	user, type, title, body, link, order, readAt	Kept for 90 days (TTL)
PushSubscription	user, endpoint (unique), keys, expirationTime, device, lastSuccessAt, failures	`toJSON` hides `keys` and `user`
Review / **ReviewReport**	target (restaurant, food or delivery), customer, order, restaurant, foodItem, deliveryPartner, rating, comment, images, status, edits, response, openReports, moderationLog / review, reporter, reason, status	Unique on `(order, target, foodItem)`
SupportTicket	number, raisedBy, requesterRole, type, subject, order, priority, status, assignee, messages (with internal notes), history	
AuditLog	admin, adminRole, action, description, resourceType, resourceId, ip	
CoinWallet / **CoinTransaction**	wallet: available, reserved, totals, seq, batches / ledger: seq, type, source, coins, allocations, idempotencyKey (unique), status	Ledger rows are immutable
Referral / **ReferralRewardCounter**	referrer, referred (unique), code, status, qualifyingOrder, riskFlags, decision, history	
RewardCampaign / **RewardClaim** / **RewardConfiguration**	campaigns with trigger, rewardCoins, dates, conditions, limits, stats / claims with claimKey (unique) / one settings document	
NewsletterSubscription	email (unique), status, source, subscribedAt	

Rating summaries (`rating-summary.schema.js` : `average` , `count` , `distribution[5]`) are embedded in restaurants, dishes and riders.

10. Business rules

10.1 Restaurants and menus

- **Approval:** a partner creates their restaurant, which starts as `pending` and is invisible to customers. An admin then moves it along the allowed transitions:
  - pending → approved or rejected
  - approved → suspended
  - suspended → approved
  - rejected → approved
- **Reasons:** rejecting or suspending requires a reason, which the owner sees.
- **Resubmission:** if the owner edits a rejected restaurant, it is resubmitted as `pending`.
- **Concurrent changes:** status changes are conditional updates, so two admins acting at once cannot both succeed.
- **Pausing:** partners can pause orders (`accepting-orders`).
- **Menu:** partners can mark dishes unavailable. Deleting a dish is a soft delete.
- **Discovery:** customers only see approved restaurants. Nearby search uses `$geoNear`. `distanceMeters` is **straight-line** distance.

10.2 Cart, pricing and placing an order

The server prices everything; the browser never sends prices. Rules are in `config/pricing.js`:

Line	Rule
Item total	Current menu price × quantity
Coupon	Percentage (optionally capped) or a flat amount, never more than the item total
Coins	100 coins = ₹10, within the redemption limits (§10.9)
Delivery fee	₹25 for the first 2 km, then ₹8 per extra km (rounded up), capped at ₹99. Free when the item total after the coupon is ₹499 or more.
Delivery range	Up to 15 km, straight line
Platform fee	₹5
GST	5% of the item total after the coupon

The cart page lists anything that blocks checkout: unavailable dishes, a closed or paused restaurant, a total below the restaurant's minimum, no address, or an out-of-range address.

Protections when placing an order (`POST /orders`):

- **Price changes:** the client sends `expectedTotalPaise`. If the server's total differs, the API returns `409 PRICE_CHANGED` and creates nothing.
- **Retries:** an `Idempotency-Key` header makes a retried request return the original order.
- **Double checkout:** the cart is claimed with a version-checked delete, so one cart can never become two orders.
- **Coupon limits:** coupon use is reserved with an atomic conditional increment. A redemption record enforces the per-customer limit.

- **All-or-nothing:** the order is written in a transaction, or each step is undone on failure when transactions are unavailable.
- **Snapshots:** orders keep copies of item names and prices, the address and the restaurant details, so later edits never change a past order.

10.3 Order lifecycle

`services/order-workflow.service.js` holds the single transition table `TRANSITIONS[from][to] = [actor roles]`. Any other move returns `409 INVALID_TRANSITION`. Never set `order.status` anywhere else.

From	To	Who
Pending	Confirmed, or Rejected (reason required)	Restaurant. Auto-rejected after 10 minutes without an answer.
Pending	Cancelled	Customer, admin
Confirmed	Preparing	Restaurant
Preparing	Ready for pickup	Restaurant. This starts dispatch.
Ready for pickup	Assigned	Rider accepting an offer
Assigned	Picked up	Rider
Assigned	Ready for pickup	Admin "unassign", which re-dispatches the order
Picked up	On the way	Rider
On the way	Delivered	Rider. Cash on delivery is marked paid.
Confirmed, Preparing, Ready, Assigned	Cancelled	Admin (reason required)

- **No cancelling after pickup:** nobody can cancel an order once it has been picked up.
- **Final statuses:** rejected, cancelled and delivered.
- **Concurrent changes:** each transition is a conditional update ("only if still in status X"). The loser of a race gets `409 ORDER_CHANGED`, and repeating a move that already happened is a no-op.
- **Side effects:** these run in `afterTransition` and each one is idempotent:
  - Releasing the coupon and coins.
  - Cancelling or refunding the payment.
  - Freeing the rider.
  - Recording the settlement.
  - Reward events.
  - Notifications.
  - Socket events.
- **Timeouts:** an online order is visible to the restaurant only once paid (`submittedAt`). Its response timer starts then. Unpaid online orders are cancelled after 15 minutes. Orders not picked up within 30 minutes of being ready are cancelled, refunded if paid, and the rider is freed.

10.4 Dispatch

When an order is ready, the backend offers it to **one** rider at a time (`dispatch.service.js` , tunables in `config/dispatch.js`):

1. **Eligible riders:** online, active account, not carrying or being offered another order, location reported within 15 minutes, within 10 km of the restaurant (straight line), and has not declined this order.
2. **Ranking:** nearest first in 500 m bands. Within a band, the rider assigned least recently goes first.
3. **Offer:** the rider has 90 seconds. Declining passes the order on at once. Ignoring the offer also passes it on, and sets that rider offline.

A rider is offered at most one order and carries at most one. Both are guarded by compare-and-set updates on `DeliveryPartner`.

10.5 Maps, routing and live location

- **Distances are labelled honestly:**
  - **Straight line:** used for fees and dispatch.
  - **Road estimate:** from OSRM, without live traffic.
  - **Live GPS:** the rider's actual progress, shown with its age.
- **Routes:** `GET /geo/route` (signed in) is cached for 10 minutes and throttled. When routing is unavailable it returns `503 ROUTING_UNAVAILABLE` and the map draws a dashed straight line instead.
- **Place search:** `GET /geo/search` and `GET /geo/reverse` go through Nominatim, cached for 24 hours.
- **Rider GPS:** shared about every 4 s while moving on a delivery, and at least every 15 s. When idle, about once a minute.
- **Customer map:** shows "Arriving in about N min · X km by road", plus "last update … ago" when the position is older than 90 s.
- **Explained problems:** the app tells the user about these instead of failing silently:
  - Location permission denied.
  - No GPS.
  - An imprecise (IP-based) location.
  - A dropped connection.
  - A stale rider position.

10.6 Payments

`GET /payments/methods` lists what checkout offers:

Method	When
Cash on delivery	Always. The restaurant gets the order at once; it is marked paid on delivery.
Razorpay	When `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET` are set
Test payment (simulated)	Development without Razorpay. Clearly labelled, moves no money, lets you pick success or failure. Never available in production.

Online flow:

1. `POST /orders` with `paymentMethod: "online"` creates the order *awaiting payment* (`submittedAt: null`) and returns the gateway checkout data.
2. The browser opens Razorpay, then calls `POST /payments/orders/:id/confirm`. The server verifies the HMAC signature, then fetches the payment and checks its status, order id and amount.

3. The webhook (`POST /payments/webhooks/razorpay`, verified by `X-Razorpay-Signature` over the raw body) is the backstop. Each event id is applied once.
4. Once paid, the restaurant sees the order and is notified.

Other cases:

- **Abandon:** closing or failing the payment page (`/abandon`) cancels the attempt, puts the dishes back in the cart and hides the attempt from "My orders".
- **Retry:** `POST /payments/orders/:id/start` reuses or creates the gateway order ("Pay ₹..." on the order page).
- **Refunds:** cancelling or rejecting a paid online order refunds it automatically, as does a payment that arrives after expiry. A failed refund shows **Retry refund** on the admin Payments page.

10.7 Earnings, settlement, payouts and reports

- **Settlement:** fixed on delivery in `order.settlement` (`config/earnings.js`):
  - **Restaurant:** food value after discounts, minus a 15% commission.
  - **Rider:** ₹20 per delivery plus ₹6 per started kilometre (straight line).
  - **Platform:** the rest. Coin discounts reduce only the platform's share.
- **Earnings pages:** restaurants and riders see totals, a daily chart and recent orders for 7, 30 or 90 days.
- **Payouts (finance):** delivered settlements become "owed". A payout claims them in one conditional update on `order.payouts.restaurant` / `.rider`, so no order is paid twice. Finance marks a payout paid with a bank/UPI reference. Cancelling a pending payout makes its orders due again.
- **Reports** (`GET /admin/reports`):
  - Figures: GMV, platform revenue, delivered and cancelled orders, average order, refunds and new customers.
  - Charts: daily trends, payment mix and top restaurants.
  - Scope: figures cover orders placed in the period. Values count delivered orders only. Days are grouped in `REPORT_TIMEZONE`.

10.8 Reviews and ratings

Policy is in `config/reviews.js`.

- **What can be reviewed:** a customer can review the restaurant, each dish and the rider of their own **delivered** order, once each.
  - **Rating:** 1 to 5 whole stars, with an optional comment of up to 1000 characters.
  - **Photos:** restaurant and dish reviews take up to 5 photos (JPEG, PNG or WebP, 2 MB each). Only URLs that BiteBox stored itself are accepted.
- **Rate in one go:** `POST /orders/{id}/reviews` saves the restaurant, food-quality and delivery ratings together, all or nothing. The comment and photos belong to the restaurant review. The food-quality rating applies to every dish not yet rated.
- **Time limits:**
  - **Writing:** within 30 days of delivery.
  - **Editing:** within 30 days of posting. Earlier versions are kept for moderators.
  - **Deleting:** final; the review cannot be written again for that order.
- **Visibility:** others see the author's full name and a "Verified order" badge. Restaurants also see the order number. Nobody sees phone numbers or addresses.

- **Restaurant replies:** restaurants post one public reply per review, which they can edit or remove. They cannot change the review itself.
- **Reports:** customers can report any review except their own. Restaurants and riders can report reviews about them.
- **Moderation** (`reviews:moderate`): **hide**, **remove** or **restore** a review, or dismiss its reports. A reason is always required; the author sees it and it is added to the moderation log. Admins never edit a review's words.
- **Ratings:** `rating { average, count, distribution }` is rebuilt from published reviews after every change (`review.service#refreshRating`), so counts cannot drift.
- **Analytics:** averages per target, daily volume, report reasons, moderation counts, and the best- and worst-rated restaurants and riders (at least 3 reviews).

10.9 Coins, referrals and special rewards

The super admin configures the programme at `/admin/rewards/*` . Defaults are in `config/rewards.js` ; the design is in `docs/COINS_REWARDS_PLAN.md` .

Rule	Default
Order reward	10 coins per ₹100 of food value after discounts, on delivered orders; at most 500 per order
Review reward	5 coins for the restaurant review, once. Taken back if the review is removed or deleted.
Redemption	100 coins = ₹10. Minimum 100 coins; at most 2,000 coins and 20% of the food value; orders of ₹199 or more
Expiry	180 days per source (0 = never). Coins that expire soonest are spent first. Returned coins stay valid for at least 7 more days.
Referral	Referrer 50 coins, friend 25, once the friend's first order (₹199+) is delivered within 30 days. Up to 50 referrals per customer and 10 rewarded per month.
Birthday / welcome	100 coins on the birthday (once a year); 25 on sign-up or the first delivered order

- **Ledger** (`coin-wallet.service.js` , the **only** writer):
  - Every change is an immutable `CoinTransaction` .
  - Each operation needs a deterministic idempotency key (for example `order-reward:<orderId>`).
  - A per-wallet sequence number serialises operations with or without transactions, so a balance can never go negative or be paid twice.
  - `reconcile()` replays a wallet's ledger to check it.
- **Checkout:**
  - `PUT /cart/coins` requests coins, and the quote decides how many apply.
  - Placing the order **reserves** them.
  - Cash orders spend them at once; online orders spend them when paid.
  - Rejected, cancelled or expired orders return them.
- **Referrals:**
  - Codes look like `BITEBOX-NAME1234` .
  - Fraud signals **hold** a referral for an admin to approve or reject: the same sign-up network, delivery to the referrer's address, a shared address, or a burst of sign-ups. Customers never see these signals.
- **Campaigns:**

    - **How they pay:** either claimed by the customer or earned automatically on qualifying orders.
    - **Targeting:** all, new, existing or inactive customers, a minimum order, first order only, or selected restaurants.
    - **Limits:** per customer, in total, and a coin budget, all enforced atomically.
    - **Statuses:** draft, scheduled, active, paused, expired and completed.
- **Events:** order, review and sign-up events reach the programme only through `reward-events.js`. This is best-effort and never fails the caller.

10.10 Support tickets

- **Who raises tickets:** customers, restaurants and riders (`/support/tickets`). They can attach their own orders.
- **Statuses:** open, in progress, waiting for response, resolved and closed.
- **Priorities:** low, medium, high and urgent.
- **Staff work:** support staff assign and reply, add **internal notes** (never shown to requesters), and escalate refund or payment issues to finance.
- **Requesters:** see staff as "BiteBox Support" and can reopen a resolved ticket by replying.
- **History:** every change is kept in the ticket history.

10.11 Operations and the audit log

- **Operations:**
    - Order, restaurant and rider counts.
    - A live board of active orders that flags ones stuck too long in a status.
    - Restaurant availability and live load.
    - Delivery performance: time per stage, by day, and top riders.
- **Audit log:** records the admin, role, action, description, record and time. It covers approvals, cancellations, unassignments, account changes, refunds, payouts, moderation, ticket changes, reward settings and push promotions. Writing it never fails the action itself.

10.12 Newsletter

`POST /newsletter/subscriptions` (public, rate limited) stores an email from the site footer. Repeating it is harmless.

11. Realtime (Socket.IO)

Socket.IO is attached to the API's HTTP server and served at `/socket.io`.

- **Handshake:**
  - The client sends `auth.token` (the access token).
  - The server verifies it, then reloads the user to check `status` and `tokenVersion`.
  - The socket is disconnected when the token expires. The client refreshes its token and reconnects.
- **Rooms** come only from the server-side identity:

Room	Who joins it
`user:<id>`	Every connected user
`restaurant:<id>`	The restaurant's owner
`admins`	Admins who have `orders:view`
`order:<id>`	Joined with `track:join` after a database permission check

Events:

Event	Direction	Payload and purpose
`order:updated`	server → client	Order id, number and status. Screens then reload over REST, so MongoDB stays the source of truth.
`notification:new`	server → client	A new in-app notification (bell count)
`track:join` / `track:leave`	client → server	Start or stop watching an order. The reply includes the last known rider position.
`rider:location`	rider → server	A GPS fix, checked as described below
`order:location`	server → order room	The rider's position, only while the order is assigned, picked up or on the way

Checks on `rider:location`:

- It must come from the rider's own connection.
- The coordinates must be valid, with accuracy within 1 km.
- No jumps faster than about 250 km/h.
- At most about one update per second, in bursts of up to five.

Everyone leaves an order's room when the order finishes.

Fallback: every live screen also polls slowly (every 30 to 60 s, paused on hidden tabs), so a dropped socket delays updates but never loses them. Browsers only play the restaurant's new-order chime after the page has been clicked once.

12. Notifications and Web Push

`notification.service.js#notify(userIds, { type, title, body, link, order })` does three things:

1. Stores the notification (kept 90 days).
2. Emits `notification:new`.
3. Hands it to `push.service.js#pushToUsers`.

It is best-effort and never throws. `notifyAdmins(payload, permission)` reaches the super admin plus every admin role that has that permission.

Who hears what:

Who	Notifications
Customers	Order placed, accepted or rejected; being prepared; rider assigned; picked up; out for delivery; delivered; cancelled; refund processed; review replies and moderation; support replies; coins earned, returned or rewarded; referral rewards; promotions (if opted in)
Restaurants	New orders (cash, or paid online), rider assigned, picked up, delivered, cancelled; restaurant approved, rejected or suspended; new reviews
Riders	Delivery requests; an order cancelled or reassigned; new ratings; support replies
Admins	New orders, rejections, cancellations, deliveries, refunds and failed refunds, restaurant applications, partner sign-ups, first report on a review, new and escalated support tickets, referrals held for review

The messages are defined in `order-notifications.js` (orders and payments), `restaurant.service.js`, `auth.service.js` and the review, support and reward services.

Pages: each panel has a bell in the header and a full Notifications page: `/notifications`, `/restaurant/notifications`, `/delivery/notifications` and `/admin/notifications`. The page has All/Unread tabs, paging, mark all read and clear all.

Web Push (`push.service.js`):

- **When it is on:** only when both VAPID keys are set. The public key is served by `GET /push/public-key`; the private key never leaves the API.
- **Categories:** each person chooses which categories they receive.

Category	Types	Default
`orders`	`order.*`, `payment.*`	on
`account`	everything else	on
`promotions`	`promo.*` (customers only)	**off**

- **Payload:** uses the Angular service worker format, so a click opens the notification's link in the app without custom worker code.
  - Only same-site links are allowed.
  - Updates about the same order replace each other on the device.
  - Order pushes are high urgency with a 1 hour TTL; others last 24 hours.

- **Subscriptions:**
  - Tied to the signed-in account and validated.
  - Limited to 10 per person and to known push services (`PUSH_ALLOWED_HOSTS`).
  - Their keys are never returned.
  - Deleted on 404/410 responses, or after 5 failures in a row.
- **Sending:** batches of 10 run in the background. Suspended users and people who switched a category off receive nothing.
- **Browser side:**
  - Permission is only requested from the "Turn on notifications" button.
  - Signing out unsubscribes the browser. Signing back in on the same browser resumes silently.
  - On iPhone and iPad, push only works in the Home Screen app (iOS 16.4+).
- **Promotions:** the super admin sends offers from `/admin/promotions` (`POST /admin/push/promotions`, `promotions:send`, rate limited and audited) to customers who opted in.
- **Tests:** swap in a fake sender with `usePushSender`, then `await flushPush()`.

13. Progressive Web App

The service worker only runs in **production builds** (`npm run build`, then `npm run serve:ssr:frontend`), on HTTPS or `localhost`.

- **Install:**
  - `public/manifest.webmanifest` provides the name, standalone display, colours, 192/512 icons and maskable versions, and shortcuts.
  - `install-prompt.service` keeps the browser's `beforeinstallprompt` event for BiteBox's own **Install** button. The button appears as a floating card on the home and restaurant list pages, and as "Install app" in the account menus.
  - "Not now" hides the card for 30 days. After `appinstalled`, the app never asks again.
  - Safari and Firefox users install from the browser's own menu.
- **Caching** (`ngsw-config.json`):
  - **App shell:** `index.csr.html`, all JS and CSS, and a few icons are precached, so the app opens offline.
  - **Assets:** images, fonts and UI icons are cached as they are used.
  - **API:** only the public catalogue (`/restaurants`, `/categories`, `/foods`) is cached. It is network-first, with a 5 s timeout and a 1-day limit.
  - **Never cached:** auth, cart, orders, payments, push and any panel or admin call. Never add a private endpoint to a data group.
- **Updates:** `app-update.service` checks for a new version:
  - When the app settles, every 6 hours, and when the tab comes back (at most every 30 minutes).
  - A "New version available" card offers **Update now** or **Later**.
  - An unrecoverable worker state shows a reload card.
- **Offline:**
  - `connection-status.service` drives an offline pill and a "Back online" message.
  - Checkout and Pay are disabled offline, with an explanation. Nothing is queued.
  - A page whose code isn't cached shows `/offline`, which continues automatically when the connection returns.
- **Headers:** `ngsw-worker.js`, `ngsw.json`, `worker-basic.min.js` and `manifest.webmanifest` are served with `Cache-Control: no-cache` by `server.ts`, `vercel.json` and `public/_headers`.
- **Emergency:** if a deployed worker is broken, copy `safety-worker.js` over `ngsw-worker.js` to unregister it.
- **Bundle rule:** keep Material and NexPrime out of the eagerly loaded code (`app.ts`, eager routes, root services). NexPrime does not tree-shake and adds about 270 kB. For this reason `app-status` is `@defer`red and the offline page uses plain HTML.
- **Layout rule:** floating UI must not shift the layout (CLS).
- **Checking it:** use the Chrome DevTools Application tab (Manifest, Service workers with its Offline and Push buttons, Cache storage) and the worker's status page at `/ngsw/state`.

14. API reference

Base URL: `/api/v1`. All bodies are JSON unless noted. Authentication uses `Authorization: Bearer <access token>`.

Access labels: public, **signed in** (any role), **customer**, **restaurant**, **rider**, and an admin **permission** shown in backticks.

Health, auth and accounts

Method	Path	Access	Purpose
GET	`/health`	public	API and database health
POST	`/auth/register`	public, rate limited	Register a customer, restaurant or rider (optional `referralCode`)
POST	`/auth/login`	public, rate limited	Sign in. Returns the access token and sets the refresh cookie.
POST	`/auth/refresh`	refresh cookie + `X-Requested-With: BiteBox`	Rotate the refresh token, get a new access token
POST	`/auth/logout`	refresh cookie + CSRF header	Revoke the session
POST	`/auth/forgot-password`	public, rate limited	Email a single-use reset link. Same response whether or not the email exists.
POST	`/auth/reset-password`	public, rate limited	Reset the password with the emailed token
GET, PATCH	`/auth/me`	signed in	Current user / update own profile
POST	`/auth/change-password`	signed in	Change password (signs out every session)
GET	`/admin/users`	`users:manage`	List accounts (role, status, search, paging)
PATCH	`/admin/users/{id}/status`	`users:manage`	Suspend or reactivate
GET, POST	`/admin/admins`	`admins:manage`	List or create staff admins
GET, PATCH	`/admin/admins/{id}`	`admins:manage`	View or edit (a role change signs them out)
PATCH	`/admin/admins/{id}/status`	`admins:manage`	Activate or deactivate
GET	`/admin/audit-logs`	`audit:view`	Audit log

Catalogue

Method	Path	Access	Purpose
GET	`/restaurants`	public	Approved restaurants (search, cuisine, category, veg, sort, `lat` / `lng` / `radiusKm`, paging)
GET	`/restaurants/cuisines`	public	Cuisines on offer
GET	`/restaurants/{slugOrId}`	public	Restaurant details
GET	`/restaurants/{slugOrId}/menu`	public	Menu grouped by category
GET	`/categories`	public	Active categories
GET	`/foods`, `/foods/{id}`	public	Dish search / dish details
GET, POST, PATCH	`/my/restaurant`	restaurant	Own restaurant (create: pending; editing a rejected one resubmits it)
PATCH	`/my/restaurant/accepting-orders`	restaurant	Pause or resume orders
GET, POST	`/my/restaurant/foods`	restaurant	Menu items
PATCH, DELETE	`/my/restaurant/foods/{id}`	restaurant	Edit / soft delete
PATCH	`/my/restaurant/foods/{id}/availability`	restaurant	Available or not
GET	`/admin/restaurants`, `/admin/restaurants/counts`, `/admin/restaurants/{id}`	`restaurants:manage`	Any status, counts, details
PATCH	`/admin/restaurants/{id}/status`	`restaurants:manage`	Approve, reject, suspend, reinstate
GET, POST	`/admin/categories`	`catalog:manage`	Categories
PATCH, DELETE	`/admin/categories/{id}`	`catalog:manage`	Edit / delete an unused category
GET	`/admin/foods`	`restaurants:manage`	All menu items
PATCH	`/admin/foods/{id}/availability`	`restaurants:manage`	Set availability
DELETE	`/admin/foods/{id}`	`restaurants:manage`	Soft delete
POST	`/uploads/images`	restaurant, admin	Upload an image (multipart `image`, JPEG/PNG/WebP, 2 MB)

Location

Method	Path	Access	Purpose
GET	`/geo/search`	public, rate limited	Place search (Nominatim, cached)
GET	`/geo/reverse`	public, rate limited	Place name for coordinates
GET	`/geo/route`	signed in	Road route, distance and duration (OSRM)

Cart, addresses, coupons and orders (customers)

Method	Path	Purpose
GET, DELETE	`/cart`	Server-priced quote (`?addressId=` quotes a specific address) / empty the cart
POST	`/cart/items`	Add a dish
PATCH, DELETE	`/cart/items/{foodItemId}`	Set quantity (0 removes it) / remove
POST, DELETE	`/cart/coupon`	Apply / remove a coupon
PUT, DELETE	`/cart/coins`	Use / stop using coins
GET, POST	`/addresses`	Saved addresses (max 10; the first becomes the default)
PATCH, DELETE	`/addresses/{id}`	Edit / delete
PATCH	`/addresses/{id}/default`	Make default
GET	`/coupons/available`	Coupons on offer
POST	`/orders`	Place an order (`Idempotency-Key` header, `expectedTotalPaise` , `paymentMethod`)
GET	`/orders` , `/orders/{id}`	My orders / one order (with the rider once assigned)
POST	`/orders/{id}/cancel`	Cancel before the restaurant accepts

Admin coupons (`catalog:manage`): `GET/POST /admin/coupons` , `PATCH/DELETE /admin/coupons/{id}` .

Restaurant and rider operations

Method	Path	Access	Purpose
GET	`/my/restaurant/orders`	restaurant	`?view=active|history`
GET	`/my/restaurant/orders/{id}`	restaurant	One order (no customer phone or exact address)
PATCH	`/my/restaurant/orders/{id}/status`	restaurant	Accept, reject, preparing, ready

Method	Path	Access	Purpose
GET	`/my/restaurant/earnings`	restaurant	Earnings (`?days=`)
GET	`/delivery/me`	rider	Profile, current offer, active delivery
PUT	`/delivery/me/vehicle`	rider	Vehicle (needed before going online)
PATCH	`/delivery/me/availability`	rider	Online (with location) / offline
PUT	`/delivery/me/location`	rider, rate limited	Report position
POST	`/delivery/offers/{id}/accept` , `/decline`	rider	Answer an offer
PATCH	`/delivery/orders/{id}/status`	rider	Picked up → on the way → delivered
GET	`/delivery/orders/history`	rider	Finished deliveries
GET	`/delivery/earnings`	rider	Earnings (`?days=`)

Admin: orders, riders, payments, reports

Method	Path	Permission	Purpose
GET	`/admin/orders` , `/admin/orders/{id}`	`orders:view`	Orders with counts / one order with rider and offer
POST	`/admin/orders/{id}/cancel`	`orders:manage`	Cancel (not after pickup)
POST	`/admin/orders/{id}/unassign`	`orders:manage`	Take back from the rider and re-dispatch
GET	`/admin/delivery-partners`	`riders:manage`	Riders with availability
PATCH	`/admin/delivery-partners/{id}/status`	`riders:manage`	Suspend or reactivate (not while carrying an order)
GET	`/admin/payments`	`finance:view`	All payments (search by order number or customer)
POST	`/admin/payments/{id}/refund`	`refunds:manage`	Retry a failed refund
GET	`/admin/reports`	`finance:view`	Platform analytics (`?days=`)

Payments

Method	Path	Access	Purpose
GET	`/payments/methods`	public	Methods offered at checkout
POST	`/payments/orders/{orderId}/start`	customer	Create or reuse the gateway order
POST	`/payments/orders/{orderId}/confirm`	customer	Confirm Razorpay (signature and amount verified)

Method	Path	Access	Purpose
POST	`/payments/orders/{orderId}/abandon`	customer	Cancel the unpaid order, restore the cart
POST	`/payments/orders/{orderId}/mock`	customer, development only	Finish a simulated payment
POST	`/payments/webhooks/razorpay`	HMAC signature	Gateway webhook (raw body)

Operations, finance and support (admin)

Method	Path	Permission
GET	`/admin/operations/dashboard`, `/monitoring`, `/restaurants`, `/performance`	`operations:view`
GET	`/admin/finance/dashboard`, `/transactions/{id}`, `/commission`, `/refunds`	`finance:view`
GET	`/admin/finance/payouts/due`, `/admin/finance/payouts`	`payouts:manage` (plus `finance:view`, required for every `/admin/finance` route)
POST	`/admin/finance/payouts`, `/payouts/{id}/paid`, `/payouts/{id}/cancel`	`payouts:manage`
GET	`/admin/support/dashboard`, `/staff`, `/tickets`, `/tickets/{id}`	`support:manage`
PATCH	`/admin/support/tickets/{id}` (status, priority, assignee)	`support:manage`
POST	`/admin/support/tickets/{id}/messages` (reply or internal note), `/escalate`	`support:manage`

Requesters (customer, restaurant or rider) use `GET/POST /support/tickets`, `GET /support/tickets/{id}` and `POST /support/tickets/{id}/messages`.

Reviews

Method	Path	Access	Purpose
GET	`/restaurants/{slugOrId}/reviews`, `/foods/{id}/reviews`	public	Published reviews with summary (`rating`, `sort`, paging)
GET, POST	`/orders/{id}/reviews`	customer	What can be rated / rate the whole order at once
POST	`/uploads/review-images`	customer	Upload a review photo
POST	`/reviews`	customer	Review one target
GET	`/reviews/mine`	customer	My reviews

Method	Path	Access	Purpose
GET, PATCH, DELETE	`/reviews/{id}`	customer	One review / edit (30 days) / delete (final)
POST	`/reviews/{id}/reports`	customer, restaurant, rider	Report a review
GET	`/my/restaurant/reviews/orders`, `/orders/{orderId}`, `/summary`	restaurant	Reviews by order, one order, summary
PUT, DELETE	`/my/restaurant/reviews/{id}/response`	restaurant	Public reply
GET	`/delivery/reviews`	rider	My ratings
GET	`/admin/reviews/orders`, `/orders/{orderId}`, `/analytics`, `/{id}`	`reviews:moderate`	Queue, details, analytics
POST	`/admin/reviews/{id}/moderation`, `/{id}/reports/dismiss`	`reviews:moderate`	Hide / remove / restore, dismiss reports

Coins, referrals and rewards

Method	Path	Access	Purpose
GET	`/coins/wallet`, `/coins/transactions`	customer	Balance and rules / history
POST	`/referrals/validate`	public, rate limited	Check a code (never reveals the owner)
GET	`/referrals/me`, `/referrals/me/history`	customer	My code and statistics / friends invited
GET	`/rewards`, `/rewards/campaigns`, `/rewards/history`	customer	Birthday and welcome, campaigns, history
POST	`/rewards/campaigns/{id}/claim`	customer	Claim a campaign reward
PUT	`/rewards/birthday`	customer	Save date of birth (once)
GET, PUT	`/admin/rewards/configuration`	`rewards:view` / `rewards:manage`	Settings
GET, POST	`/admin/rewards/campaigns`	view / manage	Campaigns
GET, PUT	`/admin/rewards/campaigns/{id}`	view / manage	Details / edit
PATCH	`/admin/rewards/campaigns/{id}/status`	`rewards:manage`	Activate, pause, end
GET	`/admin/rewards/claims`, `/transactions`, `/transactions/{id}`, `/wallets/{id}`, `/reports`, `/referrals`, `/referrals/{id}`	`rewards:view`	Lists and reports
POST	`/admin/rewards/transactions/{id}/reverse`	`rewards:manage`	Claw back a reward

Method	Path	Access	Purpose
POST	`/admin/rewards/referrals/{id}/approve`, `/reject`	`rewards:manage`	Decide a held referral

Notifications, push and newsletter

Method	Path	Access	Purpose
GET	`/notifications`	signed in	Newest first, with unread count (`?unread=true`)
DELETE	`/notifications`	signed in	Clear all
POST	`/notifications/read-all`	signed in	Mark all read
PATCH	`/notifications/{id}/read`	signed in	Mark one read
GET	`/push/public-key`	public	VAPID public key and whether push is on
POST, DELETE	`/push/subscriptions`	signed in (POST rate limited)	Subscribe / unsubscribe this browser
GET, PUT	`/push/preferences`	signed in	Categories, devices / change switches
POST	`/admin/push/promotions`	`promotions:send` (rate limited, audited)	Push an offer to opted-in customers
POST	`/newsletter/subscriptions`	public, rate limited	Subscribe an email

Common error codes

Status	Code	When
400	`BAD_REQUEST`	A request the API cannot accept
422	`VALIDATION_ERROR`	Invalid input. `details[]` lists `{ field, message }`.
401	`UNAUTHORIZED`	Missing, expired or invalid access token. The web app refreshes once, then signs out.
403	`FORBIDDEN`	Wrong role or missing permission; also an untrusted origin on `/auth/refresh`
404	`NOT_FOUND`	Not found, or not yours
409	`CONFLICT`, `INVALID_TRANSITION`, `ORDER_CHANGED`, `PRICE_CHANGED`	Duplicates, order workflow and checkout conflicts
429	`RATE_LIMITED`	Rate limit
503	`ROUTING_UNAVAILABLE`, `PUSH_UNAVAILABLE`, geocoder errors	An external service is unavailable or not configured

15. Screens and routes

Public and customer (`/`)

Path	Screen
`/`	Home: delivery point, categories, nearby restaurants (server-rendered)
`/restaurants`, `/restaurants/:slug`	Restaurant list and restaurant page (menu, reviews)
`/dishes`	Dish search
`/checkout` (`/cart` redirects here)	Cart, address, coupon, coins, payment
`/orders`, `/orders/:id`	My orders; order detail with live tracking, payment retry and rating (`?rate=1`)
`/account`, `/account/addresses`	Profile and saved addresses
`/account/coins`, `/account/refer`, `/account/rewards`	Coins wallet, Refer & earn, special rewards
`/notifications`	Notifications and push settings
`/support`, `/support/:id`	Help and support tickets
`/login`, `/register`, `/forgot-password`, `/reset-password`	Authentication
`/restaurant/register`, `/delivery/register`	Partner sign-up
`/offline`, `/forbidden`, `**`	Offline, access denied, not found

Restaurant panel (`/restaurant` , role `restaurant`)

Path	Screen
`/restaurant`	Dashboard
`/restaurant/orders`	Live order board (new-order alert and chime)
`/restaurant/details`	Restaurant details and approval status
`/restaurant/menu`	Menu management
`/restaurant/reviews`	Ratings, reviews, replies
`/restaurant/earnings`	Earnings
`/restaurant/notifications`, `/profile`, `/support`, `/support/:id`	Notifications, profile, support

Delivery panel (`/delivery`, role `delivery`)

Path	Screen
`/delivery`	Dashboard: online switch, offers, active delivery with a route map
`/delivery/history`	Finished deliveries
`/delivery/reviews`	Ratings
`/delivery/earnings`	Earnings
`/delivery/notifications`, `/profile`, `/support`, `/support/:id`	Notifications, profile, support

Admin panel (`/admin`, any admin role; each page guarded by a permission)

Path	Permission	Screen
`/admin`	`admins:manage`	Super admin dashboard (staff are sent to their own home)
`/admin/admins`	`admins:manage`	Admin management
`/admin/audit-log`	`audit:view`	Audit log
`/admin/users`	`users:manage`	Users
`/admin/categories`, `/admin/coupons`	`catalog:manage`	Categories, coupons
`/admin/promotions`	`promotions:send`	Push promotions
`/admin/rewards`, `/settings`, `/campaigns`, `/referrals`, `/transactions`, `/history`	`rewards:view`	Coins and rewards
`/admin/operations`, `/monitoring`, `/reports`	`operations:view`	Operations dashboard, live monitoring, delivery performance
`/admin/orders`	`orders:view`	Orders
`/admin/restaurants`, `/restaurants/:id`, `/admin/foods`	`restaurants:manage`	Restaurants, approval review, food items
`/admin/delivery-partners`	`riders:manage`	Riders
`/admin/finance`, `/finance/refunds`, `/finance/commission`, `/admin/payments`, `/admin/reports`	`finance:view`	Finance
`/admin/finance/payouts/:type`	`payouts:manage`	Restaurant and rider payouts
`/admin/support`, `/support/queue/:view`, `/support/ticket/:id`	`support:manage`	Support
`/admin/reviews`	`reviews:moderate`	Review moderation and analytics

Path	Permission	Screen
`/admin/notifications`, `/admin/profile`	any admin	Notifications, profile

16. Security

- **Authentication:**
  - **Access token:** a short-lived JWT (15 min) kept only in memory, never in localStorage.
  - **Refresh token:** an opaque token in an `HttpOnly`, `SameSite=Strict` cookie (`Secure` in production) scoped to `/api/v1/auth`. Only its SHA-256 hash is stored.
  - **Rotation:** the refresh token rotates on every use. Replaying an old one revokes the whole session family. A 15-second grace window covers two tabs refreshing at once.
- **CSRF:** the cookie endpoints require `X-Requested-With: BiteBox` (which forces a CORS preflight) and an allow-listed `Origin`.
- **Passwords:**
  - **Hashing and policy:** bcrypt with 12 rounds; 8–72 bytes, with at least one letter and one number.
  - **Reset tokens:** single use, valid for 30 minutes, stored hashed.
  - **Revocation:** a reset or password change revokes every session and access token (`tokenVersion`).
- **Authorisation:** checked on every route (`authenticate` plus `authorize` or `can`). The Angular guards exist only for UX. Scope always comes from the signed-in user.
- **Input:**
  - Validated with express-validator, which drops unknown fields.
  - MongoDB operators are stripped (`mongoSanitize`).
  - JSON bodies are limited to 100 kB.
  - Uploads are checked by their real bytes (JPEG, PNG or WebP, 2 MB) and stored under random names with `nosniff`.
- **Rate limits:** sign-in and sign-up (20 per 15 min per IP), coupons, uploads, geo lookups, referral checks, review reports, newsletter, push subscriptions and promotions.
- **Headers:**
  - **API:** Helmet on every response.
  - **Web server:** `nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy` and `Permissions-Policy`.
  - **Uploads:** served with a `default-src 'none'` CSP.
- **Payments:** Razorpay signatures and amounts are verified on the server. Webhooks are HMAC-checked and applied once. The test payment cannot run in production.
- **Web Push:** subscription endpoints are limited to known push services (SSRF guard). Notification links must be same-site. The private VAPID key stays on the server.
- **Privacy:**
  - **Order views:** each role sees only what it needs (§8).
  - **Logs:** secrets are redacted.
  - **Service worker:** never stores tokens or private API responses.
- **Dependencies:** `npm audit` reports nothing in the backend. The frontend's only report is a low-severity `quill` issue inside a NexPrime editor that BiteBox doesn't use.

17. Performance and accessibility

Performance

- **Compression:** gzip/brotli on the API and the web server.
- **Caching:**
  - **Static files:** fingerprinted assets are cached for a year, and the HTML shell is never cached.
  - **Public catalogue:** reads send `s-maxage=60, stale-while-revalidate=600` for CDNs.
- **Loading:**
  - **Routes:** feature routes load lazily.
  - **Home page:** rendered on the server with its data, and hydration reuses that data.
  - **PWA code:** the first-load bundle stays small because PWA UI is deferred.
- **Database:** every list and report query has an index, and reports use single aggregation passes.
- **External services:** geo calls are cached, de-duplicated and throttled.
- **Images:** Cloudinary serves them with automatic format and quality.

Accessibility (checked with axe-core on 14 key pages across the four panels, WCAG 2.1 AA)

- **Contrast:** colours meet AA.
- **Structure:** one `h1` per page and a logical heading order. Buttons and links are named by their visible text.
- **Navigation:** "Skip to content" links and visible focus styles.
- **Live content:** live regions announce order status. The map is a labelled region, and its information is also given as text.
- **Remaining finding:** a minor empty table header inside NexPrime's chart component.

18. Testing

```
# Backend (cd backend): node:test + supertest, in-memory MongoDB, files run one at a time
npm test
NODE_ENV=test node --test --test-concurrency=1 tests/orders.test.js                    # one file
NODE_ENV=test node --test --test-name-pattern="dispatch" tests/orders.test.js         # matching tests
TEST_MONGO_URI='mongodb://127.0.0.1:27017/bitebox_test' npm test                       # real DB, name must end in _test
npm run lint

# Frontend (cd frontend): Vitest via @angular/build:unit-test
npm test -- --watch=false
npm test -- --watch=false --include src/app/core/services/rider-gps.service.spec.ts   # one spec
npm run build                                                                          # production build
```

- **Backend** (about 330 tests, one file per area): auth and sessions, every role's permissions, catalogue, cart and checkout (with and without transactions), order workflow and dispatch timeouts, sockets and tracking, payments with a fake gateway, notifications, push, earnings and reports, reviews, admin roles, coins, referrals and campaigns, uploads and validation. `tests/docs.test.js` fails if a route is missing from OpenAPI. Test helpers are in `tests/helpers.js` and `tests/rewards-helpers.js`.
- **Frontend** (about 170 tests): services, guards, routes and key components.
- **Rules:**
  - Never point tests at the development database. `TEST_MONGO_URI` must end in `_test`, because the tests wipe it.
  - Use fictional data only.
  - For browser end-to-end checks, run a throwaway API on another port against an in-memory MongoDB.
- **Coverage:** `NODE_ENV=test node --test --experimental-test-coverage "tests/**/*.test.js"`.
- **CI** (`.github/workflows/ci.yml`): on every push to `master` and every pull request, it runs backend lint and tests, then frontend tests and the production build, on Node 24.

19. Deployment

One public origin: a reverse proxy sends `/api`, `/socket.io` and `/uploads` to the API, and everything else to the Angular SSR server.

```
browser ──► nginx (:8080)
              ├── /api/  /socket.io/  /uploads/ ──► api (Express, :5000) ──► MongoDB
              └── everything else               ──► web (Angular SSR, :4000) ──► api (API_INTERNAL_URL)
```

Docker Compose (recommended)

```
cp deploy/.env.example deploy/.env        # JWT_SECRET, PUBLIC_URL, PUBLIC_HOST, optional Razorpay/VAPID/Cloudinary/SMTP keys
docker compose --env-file deploy/.env up -d --build
# open http://localhost:8080
```

Service	Image	Notes
`mongo`	MongoDB 8	Single-node replica set, so transactions are used. Volume `mongo-data`.
`api`	`backend/Dockerfile`	Production dependencies only, non-root user. Volume `uploads`. Health check: `/api/v1/health`.
`web`	`frontend/Dockerfile`	Multi-stage build, runs only the built output. Health check: `/healthz`.
`proxy`	nginx with `deploy/nginx.conf`	WebSocket upgrade for `/socket.io/`, 3 MB upload limit

Without Docker

1. **API:** `cd backend && npm ci --omit=dev && NODE_ENV=production node server.js`
2. **Web:** `cd frontend && npm ci && npm run build && node dist/frontend/server/server.mjs` (`PORT`, default 4000)
3. **Proxy:** put both behind one domain with HTTPS. `deploy/nginx.conf` is a starting point.

Static hosting (Vercel or Netlify) for the frontend

- **Build output:** `npm run build` also copies `index.csr.html` to `index.html`.
- **Proxy and deep links:** `vercel.json` (Vercel) and `public/_redirects` (Netlify) proxy `/api/*` to the API, so the refresh cookie stays first-party, and serve `index.html` for deep links.
- **CORS:** set the API's `CLIENT_URL` to the exact frontend URLs. Otherwise `/auth/refresh` returns 403 "Untrusted origin" and every reload signs the user out.
- **API on Vercel:** the API can run on Vercel too (`backend/api/index.js`, `backend/vercel.json`). Keep its region close to MongoDB Atlas (`bom1` = Mumbai).

Production checklist

- ☐ HTTPS in front. Cookies are `Secure`, and service workers and push need HTTPS.

- [ ] `NODE_ENV=production`, a strong `JWT_SECRET`, `TRUST_PROXY=1` behind the proxy.
- [ ] `CLIENT_URL` and `PUBLIC_BASE_URL` set to the public URL (API); `API_INTERNAL_URL` and `NG_ALLOWED_HOSTS` set (web).
- [ ] MongoDB as a replica set (Atlas or compose), with backups.
- [ ] Cloudinary, or a persistent `uploads` volume.
- [ ] Razorpay **live** keys and the webhook at `/api/v1/payments/webhooks/razorpay` (events `payment.captured`, `payment.authorized`, `payment.failed`) with `RAZORPAY_WEBHOOK_SECRET`.
- [ ] SMTP for password-reset emails.
- [ ] VAPID keys, kept the same across deploys. Changing them invalidates every subscription.
- [ ] `ngsw-worker.js`, `ngsw.json`, `worker-basic.min.js` and `manifest.webmanifest` served with `Cache-Control: no-cache`, including on any CDN.
- [ ] A production map tile provider, geocoder and OSRM server. The public ones are for light use only.
- [ ] Never run the seed in production (it refuses to).

20. Conventions for contributors

- **Phases:** work is delivered in phases. Implement the current one, report, and wait for approval.
- **Order status:** change it only through `order-workflow.service#transition`.
- **Coin wallets:** change them only through `coin-wallet.service`, with an idempotency key.
- **Payment state:** change it only through `payment.service`.
- **Notifications:** go through `notify()`.
- **Money:** integer paise, computed on the server.
- **Multi-document writes:** `withTransaction`, plus undo steps for the fallback path.
- **New routes:**
  - Validate input with `validate(...)`.
  - Guard admin routes with `can(PERMISSION)`.
  - Add `audited(...)` to admin routes that change data.
  - Add the route to `backend/docs/openapi.*.js`; the docs test enforces this.
  - Add a test in `backend/tests/`.
- **Angular code:**
  - Standalone `OnPush` components with separate `.ts`, `.html` and `.scss` files.
  - Signal Forms with NexPrime inputs, not Reactive Forms.
  - Keep browser APIs inside `isPlatformBrowser` / `afterNextRender`.
- **Live screens:** react to socket events, and back them with `liveRefresh` polling. Requests passed to it must catch their own errors.
- **Toggles:** bind `np-toggle [checked]` to the source state, update optimistically, and restore on failure.
- **Secrets:** never in the frontend environments, never committed, never logged.
- **Sample data:** fictional only.
- **Process cleanup:** stop processes with a pattern that cannot match your own shell (e.g. `awk '/[s]erver.mjs/'`).